

Глубокий технический и архитектурный аудит Selenium → Playwright AST Migrator

Executive summary и вердикт

Вердикт: текущему Migrator нельзя доверять как функции

$$M(S, C, E) \rightarrow (T, R, V)$$

в строгом смысле воспроизводимой, детерминированной и sound миграции.

При этом проблема не сводится к «нескольким багам». В production path одновременно присутствуют несколько архитектурных свойств, которые разрывают связь между:

1. тем, что было распознано;
2. тем, что было сгенерировано;
3. тем, что было проверено;
4. тем, что final gate считает проверенным;
5. тем, что autonomous loop считает прогрессом.

Это и есть основная причина, почему система может вести себя «разумно» на unit/Lab fixtures и при этом быть нестабильной на реальном проекте.

Аудит выполнен по свежему архиву исходников, а не по README/названиям/tests как источнику истины. Ключевой production path прослежен по `Migrator.Cli`, `Migrator.Core`, `Migrator.Roslyn`, `Migrator.SeleniumCSharp`, `Migrator.PlaywrightDotNet`, orchestration scripts, migration kit и `Migrator.Lab`. В предоставленном материале нет production run artifacts, а в среде анализа отсутствует требуемый .NET SDK, поэтому выводы ниже делятся на статически доказуемые дефекты, архитектурные риски и гипотезы, для которых нужны runtime artifacts. Runtime evidence не симулируется.

Главный вывод:

Migrator сейчас ближе не к deterministic compiler, а к комбинации нескольких повторных compiler-like passes, permissive best-effort lowering, string-configured transformation DSL, независимых verification passes и agent-driven heuristic repair loop.

Это существенно больше state space, чем необходимо.

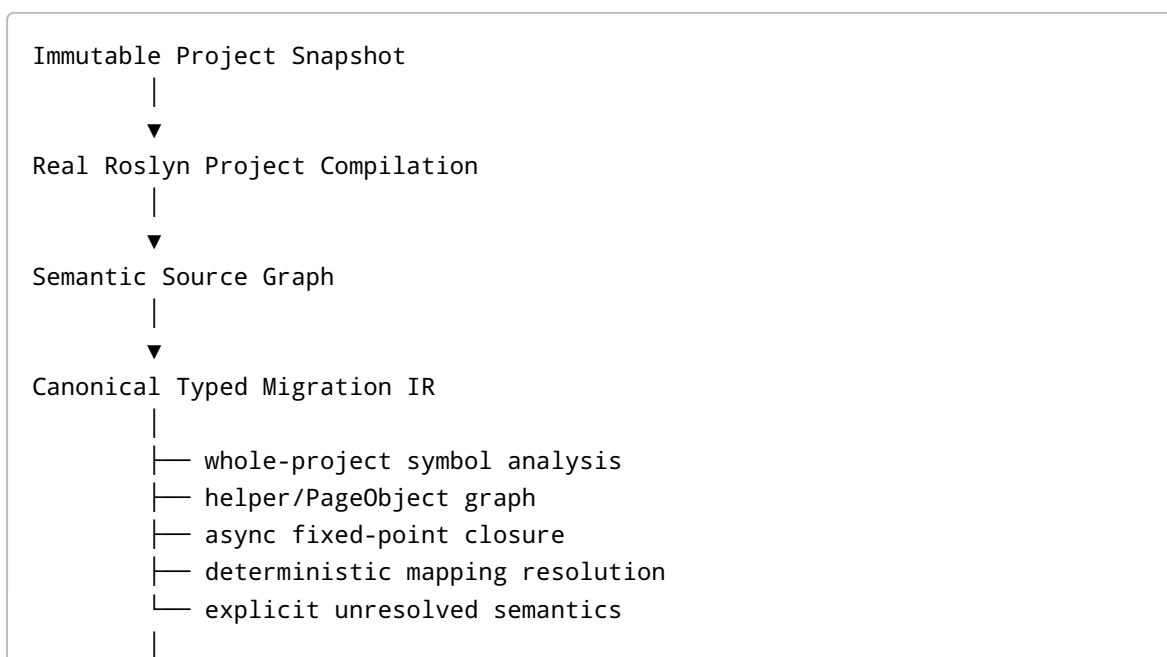
Особенно важны следующие результаты аудита.

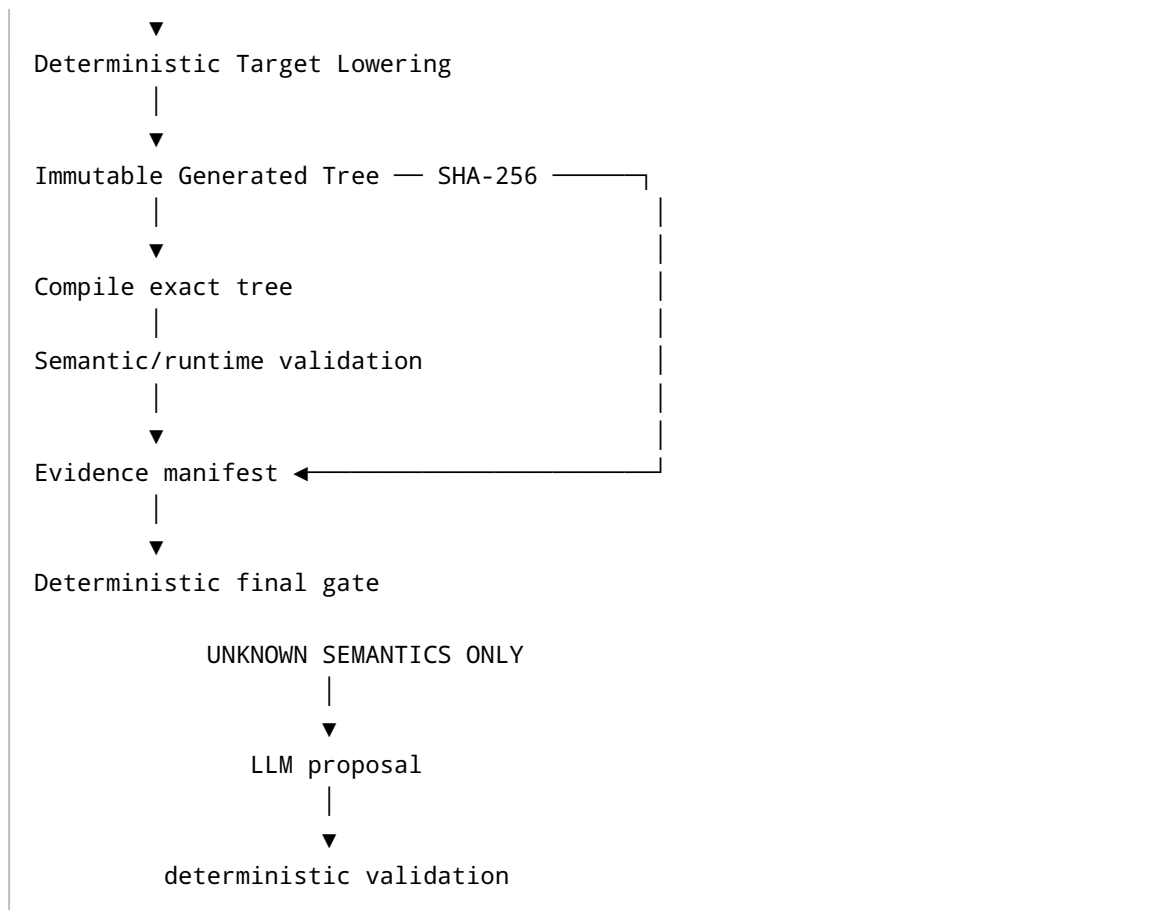
Вывод	Степень доказанности	Тяжесть
<code>orchestrate</code> не передаёт один immutable migration result через стадии: Analyze, Migrate и Verify повторно запускают pipeline	CONFIRMED DEFECT / DESIGN FLAW	Critical
Verify stage проверяет заново сгенерированный in-memory result, а не точные файлы, записанные Migrate stage	CONFIRMED DEFECT	Critical
<code>verify-project</code> также регенерирует отдельное дерево вместо проверки exact orchestration output	CONFIRMED DEFECT	Critical
final gate не связывает source/config/generated/verification hashes и может объединить логически несовместимые artifacts	CONFIRMED DEFECT	Critical
default <code>VerifyRunner</code> допускает фактически неограниченные TODO/unmapped/unsupported/raw, если quality gates явно не настроены	CONFIRMED SOUNDNESS DEFECT	Critical
source discovery использует несортированный filesystem enumeration; официальный .NET contract не гарантирует порядок <code>Directory.GetFiles</code>	CONFIRMED ORDER-SENSITIVE DEFECT	High
output naming при одинаковых class names зависит от порядка результатов	CONFIRMED ORDER-SENSITIVE DEFECT	High
generated directories в обычном migrate/orchestrate path не являются immutable/clean snapshots; stale files могут пережить новый run	CONFIRMED STATE-SENSITIVE DEFECT	Critical
scope matcher рекламирует glob-like patterns, но реализация не реализует полноценную семантику <code>*</code> / <code>**</code> / <code>*</code>	CONFIRMED DEFECT	High
compatibility mode multiple scopes → «взять первый» превращает порядок config в скрытую семантику	HIGH-RISK DESIGN FLAW	High
prefix-based target resolution выбирает первый подходящий mapping вместо longest/specific/ambiguous resolution	CONFIRMED ORDER-SENSITIVE DEFECT	High
Roslyn parser строит semantic model не из реального project compilation, а из одного source file и минимального reference set	ARCHITECTURAL LIMITATION	Critical
recognizer arbitration — first match wins без обнаружения ambiguity	HIGH-RISK DESIGN FLAW	High
parser выбирает один test class из файла и первый setup	ARCHITECTURAL LIMITATION / CONFIRMED BEHAVIOUR	High
parser catch-all на directory migration может пропустить source file после exception и продолжить	CONFIRMED SOUNDNESS DEFECT	Critical

Вывод	Степень доказанности	Тяжесть
текущая legacy IR не содержит достаточной whole-project semantics для helper/call-graph/async propagation	ARCHITECTURAL LIMITATION	Critical
IR V2 в .NET path проходит Legacy → V2 → Legacy → renderer	CONFIRMED ACCIDENTAL COMPLEXITY	High
bridge не является полностью lossless representation	CONFIRMED DEFECT/ LIMITATION	High
autonomous remediation не имеет формально вычисляемой monotonic progress metric	CONFIRMED DESIGN FACT	Critical
PROGRESS , candidate fingerprint и semantic interpretation существенным образом определяются agent-side	HIGH-RISK DESIGN FLAW	Critical
continuous remediation может продолжаться по budget rollover; convergence не доказана	CONFIRMED BUDGET TERMINATION	High
StandardMigrationModeTests доказывают в основном наличие/отсутствие текстовых contracts, а не runtime invariant	TEST GAP	High
Lab значительно сильнее обычных contract tests, но в основном работает на clean declared scenarios и не моделирует production stale state	TEST GAP	High

Есть и хорошая новость: **основная проблема исправима не расширением Migrator, а его сокращением.**

Рекомендуемая стратегия — **Architecture B с небольшим детерминированным fixed-point repair механизмом из Architecture C:**





То есть agent остаётся полезным, но меняется его роль:

LLM предлагает новую информацию или patch. Core решает, является ли patch допустимым, улучшает ли он состояние и можно ли объявить PASS.

Agent не должен:

- сам определять `PROGRESS`;
- сам определять completion;
- интерпретировать final gate;
- создавать validation evidence;
- решать, можно ли использовать stale run;
- классифицировать compiler infrastructure failure по свободному тексту;
- поддерживать собственную альтернативную модель run state.

Наиболее полезное сокращение архитектуры — удалить или постепенно вывести из production:

- повторную миграцию в verification;
- latest-run/stale-evidence semantics final gate;
- multiple-scope first-match compatibility;
- legacy↔V2 round-trip;
- duplicated scope matching;
- broad catch-and-skip;
- agent-owned progress state;

- большую часть mutable/persisted orchestration memory, которая может быть вычислена из immutable run manifest;
- raw C# template configuration из обычного semantic mapping path.

Ориентировочно, для **рекомендуемой Simplification architecture**, а не как механический LOC target:

Поверхность	Реалистично устранить/ схлопнуть
production migration/orchestration code	~15–25%
agent/orchestration mechanics	~45–60%
persisted run state/artifacts как независимые состояния	~50–70%
config semantic complexity	~25–40%
scripts/contracts, поддерживающие старую orchestration model	~35–50%

Это инженерная оценка по архитектурным обязанностям, а не измерение LOC; её следует использовать как direction, а не KPI.

Ключевой принцип redesign:

Не пытаться сделать каждый промежуточный механизм умнее. Сделать невозможным существование нескольких противоречащих друг другу версий одного migration result.

Для сравнения, build systems добиваются воспроизводимости не «умной интерпретацией прошлых runs», а строгой связью actions с declared inputs/outputs и изоляцией среды. Bazel прямо связывает correctness с hermetic/reproducible execution и action graph, а его caching модель предполагает, что одинаковые inputs дают одинаковые outputs. Это гораздо ближе к модели, которая нужна Migrator, чем текущая mutable orchestration state machine. ¹

Фактическая архитектура и формальные invariants

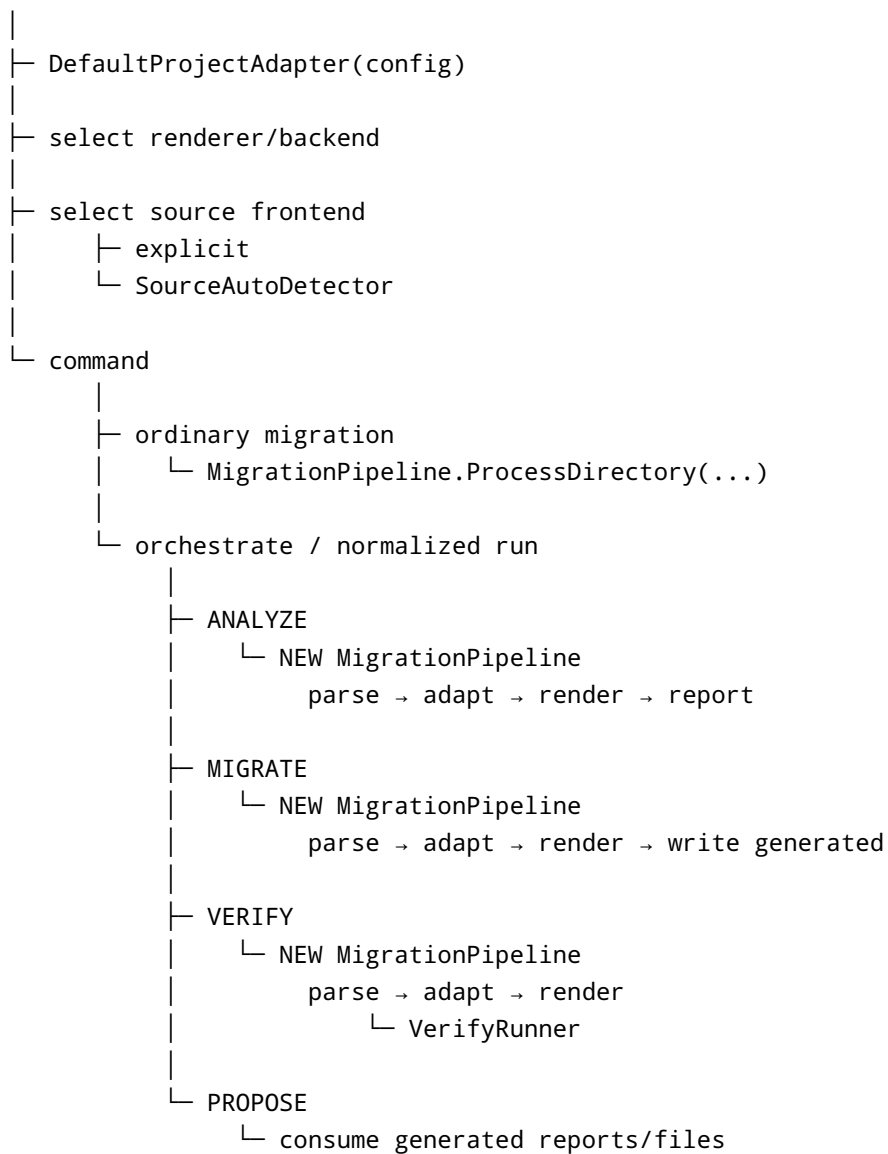
Фактический production control flow.

Основной вход находится в `Migrator.Cli/Program.cs`. Фактический путь выглядит примерно так:

```

CLI
|
├─ normalize command / args
|
├─ load config layers
|   └─ ProjectAdapterConfigMerger.LoadAndMerge(...)
|
└─ apply CLI config overrides

```



Ключевые участки: `Migrator.Cli/Program.cs`, примерно `163-239`, `505-533`, `9172-9431`.

Это даёт важный ответ на вопрос о `StandardMigrationModeTests`.

Старый partition/wave runtime действительно больше не является production execution mechanism в том виде, который проверяют эти tests. Но утверждение «production path — один linear standard migration pipeline» корректно только на уровне **stage ordering**, а не data identity.

Фактически это:

```

Parse/Adapt/Render #1
    ↓
Parse/Adapt/Render #2
    ↓
Parse/Adapt/Render #3

```

а не:

```
Parse once
  ↓
Immutable migration result
  ↓
Analyze → Write → Verify exact same result
```

`StandardMigrationModeTests.cs` проверяет строки в `Program.cs`, отсутствие старых файлов и наличие policy phrases. Это полезный architecture contract, но это не executable proof идентичности artifacts, determinism или convergence.

`Migrator.Core/Orchestrator.cs` **особенно показателен**: несмотря на название, production orchestration там практически нет; файл содержит stage status и `PathSanitizer`. Реальная orchestration логика сосредоточена в большом `RunOrchestrate` внутри CLI. То есть архитектуру действительно нельзя восстанавливать по именам классов.

Core pipeline.

`Migrator.Core/MigrationPipeline.cs` в legacy path имеет простую форму:

```
IParser.Parse
  ↓
IProjectAdapter.Adapt
  ↓
ITargetRenderer.Render
  ↓
ReportBuilder
```

Это хороший локальный дизайн. Проблема возникает вокруг него и в ограниченности models.

IR V2 выбирается отдельно через render mode. Production default остаётся legacy. Для .NET V2 path:

```
Legacy TestFileModel
  ↓
LegacyIrBridge.ToDocument
  ↓
MigrationDocument V2
  ↓
PlaywrightDotNetBackend
  ↓
bridge back to legacy
  ↓
PlaywrightDotNetRenderer
```

Следовательно, V2 сейчас не canonical production representation, а compatibility representation.

Формальная спецификация корректности.

Предлагаю считать единственным допустимым contract:

$$M(S, C, E) = (T, R, V)$$

где:

- **S** = immutable normalized source-project snapshot;
- **C** = immutable normalized configuration snapshot;
- **E** = declared environment fingerprint;
- **T** = immutable generated target tree;
- **R** = deterministic structured migration diagnostics;
- **V** = validation evidence, привязанное к exact **T**.

При этом **S**, **C**, **T** должны иметь content identifiers:

```
sourceId = SHA256(canonical S)
configId = SHA256(canonical C)
targetId = SHA256(canonical T)
```

a verification:

```
verificationId =
  SHA256(
    sourceId,
    configId,
    targetId,
    toolId,
    environmentId,
    verificationSpecification
  )
```

Необходимые invariants.

Invariant	Статус сейчас
одинаковые S, C, E → одинаковый canonical T	не обеспечен
report является функцией exact T /analysis	не полностью
verify проверяет exact T	нарушен orchestration architecture
PASS требует compile success	не обеспечен обычным VerifyRunner/ orchestrate
PASS требует completeness policy	не обеспечен defaults
persisted evidence принадлежит sourceId/ configId/targetId	не обеспечено

Invariant	Статус сейчас
ambiguous semantic resolution не выбирается произвольно	нарушается рядом first-match mechanisms
каждый source file либо обработан, либо представлен typed failure	нарушается catch-and-skip
remediation progress вычисляется core	не выполняется
accepted remediation монотонен по declared metric	не выполняется
revisiting equivalent state обнаруживается	не обеспечено
один production run имеет один authoritative target snapshot	не выполняется
continuation не меняет logical budget semantics без explicit input	не выполняется: fresh budget policy
environment dependencies перечислены	частично

Soundness здесь важнее coverage.

Корректный **PASS** должен означать минимум:

```
Parseability
AND compileability
AND no forbidden unresolved semantics
AND evidence refers to exact target tree
AND verification infrastructure successfully ran
AND final gate consumed current evidence
```

Semantic/runtime equivalence может быть отдельным, более сильным level:

```
PASS_COMPILED
PASS_SEMANTICALLY_VERIFIED
PASS_RUNTIME_VERIFIED
```

Это лучше одного перегруженного **PASS**.

Idempotence также нужно разделить на два свойства:

$$M(S, C, E) == M(S, C, E)$$

и

$$\text{render}(\text{parse}(\text{normalize}(T))) \approx T$$

Второе для Selenium→Playwright migrator не обязательно означает, что Playwright project снова должен проходить Selenium migration. Практически важнее **rerun-idempotence на одном source snapshot и workspace**: stale files, previous reports и continue state не должны менять результат.

Failure stability требует machine-readable cause, например:

```
CONFIG_AMBIGUOUS_SCOPE
RECOGNIZER_AMBIGUOUS
SOURCE_UNSUPPORTED_SEMANTICS
TARGET_COMPILE_ERROR
ENVIRONMENT_SDK_UNAVAILABLE
EVIDENCE_TARGET_HASH_MISMATCH
```

а не переходов одной и той же причины между TODO/warning/agent repair/final failure в зависимости от того, какой layer первым её увидел.

Determinism, state, configuration и hidden fallbacks

Главный determinism defect — несортированный filesystem input попадает в semantic decisions.

`RoslynTestFileParser.ParseDirectory` использует `Directory.GetFiles(..., SearchOption.AllDirectories)` без обязательной canonical sort.

Это не безобидно. .NET официально говорит, что порядок возвращаемых `Directory.GetFiles` имён **не гарантируется** и, если порядок имеет значение, результат нужно сортировать. Документация также прямо отмечает platform-dependent case sensitivity путей. ²

В Migrator этот порядок затем способен влиять на:

- порядок `MigrationPipeline` results;
- output filenames;
- report ordering;
- «первый пример» unmapped/unsupported;
- tie ordering;
- proposal inputs;
- potentially downstream agent perception.

Особенно явный пример — `ResolveFileName` в `Migrator.Cli/Program.cs`, примерно 1603–1623.

При двух исходниках:

```
A/Login.cs    → class LoginTests
B/Login.cs    → class LoginTests
```

результат назначает примерно:

```
LoginTests.cs
LoginTests_2.cs
```

в порядке поступления.

Если traversal поменялся:

```
A → LoginTests_2.cs
B → LoginTests.cs
```

Сам output-set может выглядеть похожим, но связь source→generated file изменилась. Это **ORDER-SENSITIVE**.

Минимальное исправление — не просто `.OrderBy()`, а deterministic naming:

```
logical output identity =
  normalized namespace
  + fully qualified source symbol
  + normalized relative source path
```

Collision suffix должен выводиться из canonical identity, а не из encounter order.

Классификация основных determinism dependencies.

Механизм	Класс	Почему
<code>Directory.GetFiles</code> source discovery	ORDER-SENSITIVE	contract не задаёт order
duplicate generated class naming	ORDER-SENSITIVE	suffix зависит от encounter order
first recognizer	ORDER-SENSITIVE	precedence — list position
first matching scope compatibility	ORDER-SENSITIVE	config order определяет semantics
prefix target mapping	ORDER-SENSITIVE	первый prefix wins
config layering	DETERMINISTIC, если order объявлен input	later layer override
report top-N при равном count	ORDER-SENSITIVE	нет полного tie-break

Механизм	Класс	Почему
reused generated directory	STATE-SENSITIVE	старые files участвуют в новом state
<code>latest run</code> selection	STATE/TIME-SENSITIVE	filesystem timestamps
agent candidate naming/fingerprint	AGENT/STATE-SENSITIVE	нет canonical defect identity
<code>Guid.NewGuid()</code> для отдельных internal identities	ACTUALLY NONDETERMINISTIC, хотя обычно non-semantic	случайный runtime value
timestamps in reports	ENVIRONMENT/TIME-SENSITIVE	не функция <code>S, C</code>
<code>Environment.NewLine</code>	ENVIRONMENT-SENSITIVE	Windows/Linux bytes отличаются
path case behaviour	ENVIRONMENT-SENSITIVE	OS/filesystem semantics
external <code>dotnet build</code> /restore	ENVIRONMENT-SENSITIVE	SDK/packages/network/cache
mutable adapter cache при потенциальной parallel use	RACE-SENSITIVE latent	mutable non-thread-safe state

При этом не вся environment sensitivity является bug. Нужно просто сделать `E` explicit.

Например:

```
{
  "os": "...",
  "pathSemantics": "...",
  "dotnetSdk": "...",
  "targetFramework": "...",
  "playwrightPackage": "...",
  "locale": "en-US",
  "newlinePolicy": "LF",
  "packageLockHash": "...",
  "toolBuildHash": "..."
}
```

Build-system подход здесь полезен: hermetic execution означает, что action использует только объявленные inputs и производит объявленные outputs; именно это позволяет воспроизводимость и безопасное caching. ³

Stale generated state — один из наиболее серьезных найденных дефектов.

Обычный `RunMigrate` создаёт output directory, но не делает его immutable clean snapshot.

`RunOrchestrate` аналогично создаёт `generated` directory, затем записывает текущие generated files, не гарантируя, что files от предыдущего/частично завершённого invocation исчезли.

Сценарий:

```
Run 1:
  generated/
    LoginTests.cs
    CheckoutTests.cs

source changes

Run 2:
  actual current result:
    LoginTests.cs

  generated/
    LoginTests.cs
    CheckoutTests.cs ← stale
```

После этого Propose stage использует `Directory.GetFiles(generatedDir, "*.cs")`. Stale `CheckoutTests.cs` оказывается частью perceived current state.

Это прямое нарушение:

```
target tree = F(current source, current config)
```

Потому что фактически:

```
target tree =
  F(current source, current config)
  UNION leftovers(previous state)
```

CONFIRMED DEFECT.

Исправление:

```
generate into:
  .staging/<targetHash-candidate>/

verify staging

atomic rename/promote:
  outputs/<targetHash>/
```

Никогда не писать authoritative target «поверх» предыдущего target.

Config scope semantics.

`DefaultProjectAdapter.GetActiveScope` и per-file config resolution собирают matching scopes.

Default behaviour `FailOnMultipleMatchingScopes ?? true` является правильным направлением: ambiguity должна быть ошибкой.

No compatibility path:

```
multiple matches
→ warning
→ matching[0]
```

архитектурно неправильный.

Вопрос пользователя «почему вообще один source file может соответствовать нескольким scopes?» имеет два ответа.

Иногда overlap закономерен:

```
**/Tests/**
**/Tests/Admin/**
```

если scopes задуманы как compositional layers.

Но тогда semantics должна быть:

```
base scope
+ more-specific scope
```

с формальным partial order.

Если scopes задуманы как mutually exclusive profiles, overlap должен быть config error.

Третьего безопасного варианта — «возьмём первый» — нет.

Рекомендация:

```
ProfileScopeMode =
    Exclusive
    Hierarchical
```

В `Exclusive`:

```
matches.Count != 1 → CONFIG_SCOPE_AMBIGUOUS
```

В **Hierarchical**:

```
resolve specificity  
→ merge only if one chain  $A \subset B \subset C$   
→ incomparable overlap = error
```

Compatibility first-match — **DELETE**.

Scope glob matcher содержит отдельный дефект.

Документируемый config path допускает glob-like `**/*`, однако `MatchPathPattern` фактически не является полноценным glob engine: он специальным образом разбирает `**`, но оставшиеся `*` не интерпретируются как wildcard.

Следовательно patterns наподобие:

```
**/*.cs  
**/DiscountsTests/**
```

не получают обещанной glob semantics.

Это **CONFIRMED DEFECT**.

Ещё хуже — похожая matching logic продублирована в `VerifyRunner`, поэтому возможен semantic drift между «адаптер считает score активным» и «verifier считает score активным».

Исправление:

```
one ScopeResolver  
one compiled GlobPattern representation  
one semantics  
used by:  
  adapter  
  validator  
  verifier  
  reports
```

И config validation должна проверять pairwise overlap/specificity до migration.

Mapping lookup имеет order-dependent prefix resolution.

`DefaultProjectAdapter.ResolveTarget`, после exact matches и special cases, итерирует `_targetMap` и возвращает первый mapping, чей key является prefix.

Минимальный reproducer:

```
UiTargets:  
page.Panel -> A  
page.Panel.Button -> B
```

Для source expression, подходящего под оба prefix, переменная порядка config способна изменить mapping result.

Правильные alternatives:

```
exact match  
→ unique longest semantic prefix  
→ ambiguity error
```

но не first iteration.

Это особенно опасно, потому что `ProjectAdapterConfigMerger` использует dictionary-based merge. Сам merge при фиксированном ordered layer input достаточно предсказуем, но **dictionary insertion order не должен становиться semantic language feature**.

Config фактически перерос в слаботипизированный язык трансформаций.

`ProjectAdapterConfig` содержит не только декларативные facts вроде locator mappings, но и:

- `Methods` ;
- `ParameterizedMethods` ;
- arbitrary target statements;
- target expressions;
- setup statements;
- placeholders;
- recognizer aliases;
- suppressions;
- scaffold policies;
- target-specific templates;
- wait policies;
- verification policies.

Особенно опасны fields наподобие:

```
TargetStatements  
TargetExpression  
SetUpStatements
```

которые позволяют C# semantics кодировать строками.

Получается:


```
typed C# source
→ partly typed IR
→ untyped strings from configuration
→ C# source again
```

Это inversion of responsibility.

Config должен содержать semantic facts:

```
source symbol X corresponds to target PageObject Y
locator Z has role/button/name...
custom helper H has semantic intent WaitUntilVisible
```

но target code lowering должен быть typed:

```
Intent.WaitUntilVisible(target)
→ PlaywrightWaitLowerer
→ AwaitExpression(...)
```

а не:

```
"await {target}.WaitForAsync(...)"
```

Raw target templates можно оставить только как явно помеченный escape hatch:

```
UnsafeTargetTemplate
requiresManualReview = true
cannotProduceVerifiedPass = true
```

Hidden fallbacks.

Самые опасные:

Fallback	Verdict
source parse exception → warning + skip file	DELETE/REPLACE
one of multiple fixture classes → first class	REPLACE representation
multiple setup methods → first setup	REPLACE representation
recognizer overlap → first recognizer	REPLACE arbitration
unresolved syntax → Raw/TODO и pipeline continues	KEEP только как explicit unresolved IR
multiple scopes → first	DELETE

Fallback	Verdict
target prefix → first	FIX
absent quality limits → <code>int.MaxValue</code>	FIX
latest run by mtime	DELETE
missing verification override yielding gate success	DELETE from production success path
stale generated directory reuse	DELETE state semantics

Здесь ключевой criterion:

Fallback хорош, если он сохраняет truthfulness.

Fallback плох, если он сохраняет только способность pipeline продолжать выполнение.

`UnsupportedIntent` — хороший fallback.

```

UnresolvedIntent(
    Code = SOURCE_SEMANTICS_UNKNOWN,
    Location = ...,
    OriginalSyntax = ...
)

```

— тоже хороший.

А сгенерировать правдоподобный вызов и поставить warning — существенно хуже явной остановки этого fragment.

State model.

Сейчас независимо существуют как минимум:

```

source state
config state
generated state
analysis reports
verify state
verify-project state
proposal state
orchestration state
final-gate state
autonomy state
agent memory
handoff/current ticket state

```

Не все они плохи по отдельности. Проблема в отсутствии единого identity relationship.

Целевой invariant:

```
RunManifest {  
    runId,  
    sourceHash,  
    configHash,  
    toolHash,  
    environmentHash,  
  
    analysisHash,  
    targetHash,  
  
    compilationEvidence {  
        targetHash,  
        commandHash,  
        environmentHash,  
        resultHash  
    },  
  
    semanticEvidence {...},  
  
    finalDecision  
}
```

Каждый artifact либо referenced этим manifest, либо не является authoritative.

Никаких:

```
latest run  
latest verify  
current generated  
current evidence
```

Только:

```
artifact identified by hash
```

Recognizers, IR, async propagation и semantic modeling

Главное ограничение recognizer architecture — production frontend не является настоящим project semantic frontend.

`Migrator.Roslyn/RoslynTestFileParser.cs` читает отдельный file, строит syntax tree и затем `CSharpCompilation` с небольшим набором framework references. В него не загружается реальная solution/project compilation со всеми:

- project references;

- Selenium symbols;
- NUnit/xUnit/MSTest symbols;
- project helper types;
- extension methods;
- PageObjects;
- partial class parts;
- base classes.

Из-за этого `SemanticModel` существует технически, но значительная часть domain resolution переходит обратно в syntax heuristics.

В самом parser semantic recognition по сути ограничен небольшим набором специальных случаев, после чего идёт fixed recognizer list.

Официальная Roslyn модель разделяет syntax, workspace/symbol information и `SemanticModel`; именно semantic binding позволяет решать вопросы symbol identity, overload resolution и references, которые невозможно надёжно решить по текстовой форме receiver/method name. 4

Для source-to-source migration это фундаментально.

Фактический recognizer precedence в `RoslynTestFileParser` примерно такой:

```

WebDriverFindElement
Click
SendKeys
Assert
Table
ProjectAssertionHelper
FluentTextAssertion
Visibility
WaitPresence
UrlAssertion
FluentAssertions
WaitInvocation
AsyncPlaywright
Navigation
PlaywrightAssertion
SelectValue
PageObjectMethod
Unknown

```

В invocation path после нескольких special semantic checks выполняется:

```

foreach recognizer:
    if TryRecognize(...)
        return first result

```

Таким образом arbitration model:

$$winner = first\{R_i \mid R_i(x) = true\}$$

а должна быть ближе к:

$$Candidates(x) = \{(R_i, confidence, semanticEvidence)\}$$

после чего:

```
0 candidates → Unresolved
1 candidate → accepted
N compatible candidates → select by explicit specificity
N conflicting candidates → RecognizerAmbiguity
```

`ClickInvocationRecognizer` **показывает риск false positives.**

Syntax-level rule для receiver `.Click()` может распознать не-Selenium domain method:

```
menu.Click();
customControl.Click();
trackingObject.Click();
```

как Selenium-like click, если symbol resolution недостаточно.

Специализированные recognizers, поставленные раньше general recognizers, уменьшают вероятность collision, но **list order является не доказательством semantics.**

Нужен recognizer contract:

```
Recognized<TIntent> {
  SourceSymbol?
  RequiredFacts[]
  Confidence = ExactSymbol | ExactPattern | Heuristic
}
```

И production lowering должно разрешать автоматически только:

```
ExactSymbol
или
validated project-specific contract
```

Heuristic recognition должно быть diagnostic/proposal, а не silent active lowering.

Parser также имеет structural context loss.

Он выбирает один `testClass`:

```
first [TestFixture]
else first class containing test
else sole class
```

и один setup через `FirstOrDefault`.

Файл:

```
[TestFixture]
class LoginTests { ... }

[TestFixture]
class CheckoutTests { ... }
```

не является полноценно представимым текущим `TestFileModel`.

То же касается:

- helper methods;
- interface implementations;
- overridden methods;
- base fixture hooks;
- nested test classes;
- partial classes across files.

Это не recognizer bug. Это **IR/source-model limitation**.

Broad catch в `ParseDirectory` нарушает completeness.

Pattern примерно такой:

```
foreach file:
  try:
    Parse(file)
  catch Exception:
    warning
    continue
```

означает, что internal parser error, transient I/O failure или unexpected shape потенциально превращаются в «этого source file как будто нет».

Правильная структура:

```
SourceFileResult =  
    Parsed(FileIR)  
    | ParseFailure(ErrorCode, ...)
```

и directory result содержит **ровно один entry на каждый declared source file**.

Тогда невозможно получить silent omission.

Legacy IR против IR V2.

Сейчас две representation models не дают двух действительно независимых production implementations.

Для .NET V2 path происходит:

```
Legacy  
→ LegacyIrBridge.ToDocument  
→ V2  
→ PlaywrightDotNetBackend  
→ V2 back to Legacy  
→ existing renderer
```

Это худший transitional steady state:

- две schemas;
- две conversion directions;
- compatibility tests;
- дополнительный failure surface;
- при этом target lowering всё ещё зависит от legacy model.

Bridge также не сохраняет всю legacy semantic information при round-trip. Среди полей, которые не имеют полного symmetric representation, находятся parts of test metadata, generic invocation detail и некоторые legacy-specific attributes/config-derived details.

Следовательно:

поддерживать Legacy + V2 как две долгоживущие canonical representations не стоит.

Рекомендация — выбрать V2/evolved typed IR как единственную canonical model, но не делать migration через bridge forever.

Переход:

```
Phase A  
    introduce canonical typed nodes + invariants
```

```

Phase B
  make Roslyn semantic frontend emit canonical IR directly

Phase C
  adapter becomes semantic intent enrichment, not legacy mutation

Phase D
  direct Playwright .NET lowerer

Phase E
  run old/new in differential mode in Lab

Phase F
  remove LegacyIrBridge and legacy renderer entry path

```

Во время перехода лучше даже **убрать публичный experimental V2 production switch**, если он лишь добавляет Legacy→V2→Legacy round-trip. Он создаёт дополнительный observable mode без дополнительного correctness.

Как должна выглядеть canonical IR.

He:

```

ClickAction
SendKeysAction
RawStatementAction
MethodInvocationAction(receiverText, ...)

```

как финальная semantic boundary.

A:

```

SourceSymbolRef
SemanticTargetRef
MigrationIntent
AsyncRequirement
SideEffectModel
UnresolvedReason
SourceLocation
Evidence

```

Например:

```

LocatorIntent {
  semanticTargetId
  strategy
  value
}

```



```

}

InteractionIntent {
    kind = Click
    target = LocatorIntent(...)
}

AssertionIntent {
    subject
    predicate
    expected
    timingSemantics
}

HelperCallIntent {
    sourceMethodSymbol
    arguments
    effectSummary
}

```

Async propagation — фундаментально whole-project graph problem.

Текущий renderer просто делает generated tests/setup async и генерирует `await` там, где знает Playwright action.

Это работает для локального:

```

test
  → driver.FindElement(...).Click()

```

Но не решает:

```

Test A
  → Helper B
    → PageObject C
      → WaitHelper D
        → Selenium call

```

Если `D` становится async:

```
D : void → Task
```

то async requirement распространяется обратно:

```

D async
↑

```

```
C awaits D → C async
↑
B awaits C → B async
↑
A awaits B → A async
```

Это fixed-point closure.

Формально:

$$\begin{aligned} Async_0 &= \{m \mid \text{lowering}(m) \text{ requires await}\} \\ Async_{n+1} &= Async_n \cup \{caller \mid caller \rightarrow callee, callee \in Async_n\} \end{aligned}$$

до:

$$Async_{n+1} = Async_n$$

Но есть boundaries:

```
constructors
properties
interface contracts
overrides
event handlers
framework lifecycle methods
delegates
expression trees
external API implementations
```

Поэтому анализ должен возвращать не просто closure, а constraints:

```
MethodRequiresAsync(symbol)
BoundaryCannotChangeSignature(symbol)
CallerMustAdapt(symbol)
UnresolvedAsyncBoundary(...)
```

Текущая legacy model не содержит method bodies/call graph, поэтому такое решение **НЕВОЗМОЖНО** добавить несколькими recognizer if-statements.

Это аргумент за semantic frontend, а не за расширение текущих action recognizers.

PageObjects и helpers требуют того же перехода.

Реальный Selenium project обычно имеет промежуточный application-specific language:

```
Tests
→ PageObjects
```

- Components
- Wait wrappers
- Locator wrappers
- Assertion helpers
- Selenium

Сейчас часть этой semantics реконструируется:

- syntax recognizers;
- config mappings;
- helper inventory;
- PageObject mappings;
- agent reasoning.

То есть semantic knowledge рассредоточено.

Предлагаемая layering:

```
C# syntax
  ↓
Roslyn symbol graph
  ↓
Source semantic normalization
  ↓
Project-specific semantic summaries
  ↓
Migration Intent IR
  ↓
Whole-project constraints
  ↓
Playwright target IR
  ↓
C# lowering
```

Именно здесь лежит граница между необходимой и случайной сложностью:

Необходимая сложность:

- symbol resolution;
- helper semantics;
- async call graph;
- locator semantics;
- framework lifecycle;
- target compilation constraints.

Accidental complexity:

- несколько IR;
- string templates;

- repeated pipelines;
- mutable run state;
- first-match resolutions;
- agent state interpreting core state.

Responsibility split.

Responsibility	Сейчас	Должно быть
AST recognition	core + syntax heuristics	core
symbol resolution	partial	core, real project compilation
locator mapping	config + adapter + agent	typed config facts + core
PageObject indexing	dispersed	core
PageObject semantic unknowns	agent/config	agent proposal, core validation
helper call graph	практически отсутствует в canonical model	core
helper body migration	mappings/agent	deterministic where known, agent only unknown semantics
async propagation	local rendering	core fixed point
compile repair	scripts/agent/manual	typed deterministic diagnostics first
semantic repair	agent	agent proposal + oracle validation
config generation	agent/proposal	agent may propose; core validates
verification interpretation	verifier + scripts + agent	core only
progress calculation	agent-side	core only
stopping decision	policy + agent	core only
final PASS	scripts/agent contracts	core evidence gate only

Verification, evidence, agent remediation и termination

Сейчас слово “verification” обозначает несколько существенно разных вещей.

Нужно разделить пять уровней:

```
V1 Syntax
V2 Compilation
V3 Migration completeness
V4 Semantic equivalence
V5 Runtime correctness
```

Текущая система покрывает их неравномерно.

`VerifyRunner` в `Migrator.Core` делает:

- config/scope checks;
- syntax checks;
- TODO scanning;
- suspicious generated constructs;
- counts unsupported/unmapped/raw;
- quality gates.

Но он **не является target project compiler**.

Это означает:

```
VerifyRunner PASS
≠
generated project compiles
```

Более того, quality defaults permissive.

При отсутствии explicit quality gates:

```
MaxTodoComments = int.MaxValue
MaxUnsupported   = int.MaxValue
MaxUnmapped      = int.MaxValue
MaxRaw           = int.MaxValue
```

или эквивалентная soft-default semantics.

Следовательно syntactically valid target с большим количеством unresolved migration debt способен пройти этот verification layer.

Это можно считать допустимым для diagnostic command.

Но нельзя использовать такой status как:

```
MIGRATION_SUCCESS
```

Нужна vocabulary:

```
ANALYSIS_COMPLETE
GENERATED_WITH_UNRESOLVED
SYNTAX_VALID
COMPILES
```

```
SEMANTICALLY_VERIFIED
RUNTIME_VERIFIED
```

Самый серьёзный verification defect — verification не проверяет exact migration artifact.

В `RunOrchestrate`:

```
Migrate:
  pipeline #2
  → files A

Verify:
  pipeline #3
  → in-memory files B
  → VerifyRunner(B)
```

Никакого invariant:

```
SHA256(A) == SHA256(B)
```

нет.

Даже если сейчас pipeline *обычно* deterministic, verification architecture должна быть sound независимо от этого assumption.

Правильно:

```
Analyze source snapshot S
  ↓
produce TargetTree T
  ↓
persist immutable T
  ↓
Verify(T)
```

не:

```
Produce T1
Produce T2
assume T1 == T2
verify T2
```

`verify-project` усиливает разрыв.

Этот command правильно создаёт clean generated/harness directories, что является хорошей изоляцией.

Но он снова мигрирует source и строит **свой** generated project.

Итоговый workflow способен выглядеть:

```
run:
  generated T1

verify-project:
  regenerated T2
  dotnet build T2
  PASS

final gate:
  sees run artifacts + PASS verify-project
  → PASS
```

Без:

```
T1.hash == T2.hash
```

Это фундаментально недоверенное evidence.

Final gate.

templates/migration-kit/scripts/check-final-gate.ps1 :

- при отсутствии explicit run path может выбрать latest run по LastWriteTimeUtc ;
- проверяет наличие orchestration/generated reports;
- project verification главным образом оценивается по report status;
- имеется compatibility behaviour для missing verification;
- final gate artifact содержит timestamp/path;
- нет обязательных source/config/generated/tool/environment hashes.

Особенно важно: существование orchestration-report.json не эквивалентно доказательству, что сам orchestration был successful; gate должен разбирать authoritative manifest, а не только наличие отдельных документов.

Минимальная trustworthy evidence model:

```
{
  "schemaVersion": 1,
  "source": {
    "sha256": "...",
  },
}
```

```

"config": {
  "sha256": "...",
},
"tool": {
  "version": "...",
  "buildSha256": "...",
},
"environment": {
  "fingerprint": "...",
},
"generatedTree": {
  "sha256": "...",
},
"verification": {
  "kind": "dotnet-build",
  "commandFingerprint": "...",
  "targetTreeSha256": "...",
  "exitCode": 0,
  "diagnosticsSha256": "...",
},
"semanticVerification": {
  "targetTreeSha256": "...",
  "status": "...",
}
}

```

И final decision:

```

PASS iff
all required evidence targetTreeSha256 == manifest.targetTreeSha256
AND source/config/tool/environment identities agree
AND required stages have success status

```

Не нужен даже сложный cryptographic signing на первом этапе. **Content-addressing уже устраняет основной класс stale/mismatched evidence.**

Error taxonomy должна стать частью evidence contract.

Сейчас ошибки представлены смесью:

- exceptions;
- strings;
- Unsupported reasons;
- verifier categories;
- process exit codes;
- agent stop reasons.

Предлагаемая taxonomy:

Code family	Пример
SRC_*	SRC_PARSE_FAILED, SRC_MULTIPLE_TEST_TYPES
SEM_*	SEM_SYMBOL_UNRESOLVED, SEM_HELPER_UNKNOWN
REC_*	REC_NO_MATCH, REC_AMBIGUOUS
CFG_*	CFG_INVALID, CFG_SCOPE_AMBIGUOUS, CFG_MAPPING_AMBIGUOUS
IR_*	IR_INVARIANT_VIOLATION
LOWER_*	LOWER_UNSUPPORTED_INTENT
RENDER_*	RENDER_INTERNAL_ERROR
BUILD_*	BUILD_COMPILATION_FAILED
VERIFY_*	VERIFY_SEMANTIC_MISMATCH, VERIFY_RUNTIME_FAILED
ENV_*	ENV_SDK_MISSING, ENV_BROWSER_MISSING, ENV_RESTORE_FAILED
EVIDENCE_*	EVIDENCE_STALE, EVIDENCE_TARGET_MISMATCH
REMEDIATION_*	REMEDIATION_EXHAUSTED, REMEDIATION_CYCLE
INTERNAL_*	invariant/assertion bugs

Каждый diagnostic:

```
code
stage
sourceLocation
stableFingerprint
ownership: SOURCE|CONFIG|MIGRATOR|ENVIRONMENT
retryability
evidence
```

Это сразу решает проблему, когда infrastructure failure отправляется агенту как migration defect.

Autonomous remediation сейчас является эвристическим search loop, а не algorithm of monotonic improvement.

Policy содержит:

```
maxRemediationCyclesPerInvocation
maxChangesPerCycle
continueStartsFreshBudget
continuousAutoAdvanceAfterProgress
continuousRollsOverCycleBudget
requireDistinctNoProgressCandidates
noProgressStopThreshold
```

`update-autonomy-state.ps1` принимает извне:

```
CandidateFingerprint
Result = PROGRESS | NO_PROGRESS | BLOCKED
MetricSummary
```

Критично то, что script **не выводит** `PROGRESS` из **canonical state metrics**.

В agent instructions progress включает такие понятия, как:

- meaningful metric improves;
- blocker resolved;
- new evidence;
- etc.

Это полезная human/LLM heuristic, но не математическая progress function.

Поэтому для:

```
X0 → X1 → X2 ...
```

сейчас нельзя предъявить scalar/lexicographic `P`, для которого гарантировано:

$$P(X_{n+1}) > P(X_n)$$

для каждого accepted `PROGRESS`.

Возможен сценарий:

```
X0:
  compile errors = 3

repair A
X1:
  compile errors = 2
  semantic defects = 4
agent: PROGRESS

repair B
X2:
  compile errors = 3
  semantic defects = 2
agent: PROGRESS

repair A'
X3 ≈ X1
```

Distinct candidate fingerprint не является distinct **state**.

Ещё хуже, если fingerprint agent-formulated:

```
fix-login-helper  
repair-login-helper  
resolve-helper-login
```

могут быть тремя aliases одной operation.

Termination сейчас в значительной мере budget termination.

```
maxCycles = N
```

не доказывает convergence.

При continuous budget rollover даже конечный invocation budget превращается в batch budget.

Термины нужно разделить:

- **converged** — fixed point достигнут;
- **cycle detected** — revisited state;
- **exhausted** — candidates исчерпаны;
- **budget exhausted** — stop из-за лимита;
- **blocked** — требуется новая semantic information;
- **success** — deterministic final gate passed.

Строгая remediation model.

Определить defect vector:

$$D(X) = (I, C, S, U, M, T, W)$$

где:

- **I** — invariant/internal errors;
- **C** — compile errors;
- **S** — semantic verification errors;
- **U** — unsupported;
- **M** — unmapped;
- **T** — TODO;
- **W** — warnings.

Меньше — лучше, comparison lexicographic.

Например:

```
D1 < D0
iff
  first differing dimension in D1 is lower.
```

Можно добавить:

```
changedSourceSemantics
unsafeRawTemplates
```

выше warnings.

Accepted repair:

```
candidate C
apply in isolated workspace
→ state X'

accept iff:
  valid(X')
  AND D(X') < D(X)
  AND stateHash(X') not in visited
```

Иначе rollback.

Иногда реальный global repair требует временного ухудшения compile errors. Для этого нужен **transaction candidate**, а не принятие промежуточного плохого state:

```
agent proposes:
  edit A + edit B + edit C

validate complete patch atomically
```

Таким образом возвращается глобальная гибкость старого агента без surrender determinism.

Почему «простой агент» иногда справлялся лучше.

Парадокс реальный и объяснимый.

Старый agent мог:

```
видеть compiler error
→ открыть helper
→ изменить base fixture
→ поменять mapping
```

→ обновить PageObject
→ снова compile

то есть свободно пересекать abstraction boundaries.

Formal orchestration ввела локальные contracts:

recognizer boundary
config boundary
quality gate
one-change cycle
candidate exhaustion
state ledger

но сама formal model неполна.

Получился эффект:

реальное решение лежит за пределами formal search space

и система останавливается «правильно по своей модели», но неправильно относительно реальной задачи.

Проблема не в формализации как таковой.

Проблема в:

hard constraints поверх incomplete semantic model.

Полезное свойство старого agent, которое стоит вернуть:

ability to propose coordinated cross-layer change

Не стоит возвращать:

ability to declare its own evidence valid
ability to decide success
ability to mutate opaque run state

Целевая модель:

LLM = global proposal/search engine
Core = theorem checker for the practical invariants

Migrator.Lab, тестовая система, reproducibility и experiments

`Migrator.Lab` — одна из наиболее архитектурно ценных частей проекта. Она уже содержит многие элементы правильной будущей системы:

- sorted scenario catalog;
- clean per-scenario workspaces;
- content hashes;
- scenario contracts;
- baselines;
- normalized generated hashes;
- semantic oracle;
- source/target runtime testing;
- seeded variants;
- pairwise generation;
- metamorphic comparison;
- triage;
- saved-seed promotion;
- release gate.

Например `ScenarioCatalog` явно сортирует `scenario.json` paths через `OrderBy(..., OrdinalIgnoreCase)`, что как раз является правильным contrast с production parser.

`LabRunCoordinator` создаёт новый workspace с GUID для каждого scenario и копирует declared project files. Random workspace pathname при этом не должен влиять на semantic result; это даже хороший implicit perturbation, если canonical output excludes path.

LabSemanticOracle сильнее обычного snapshot test.

Он способен проверять:

- expected target test count;
- event sequence через Lab app observations;
- timing bound;
- final DOM conditions;
- generated structured assertions;
- expected diagnostics;
- forbidden diagnostics.

Это существенно лучше правила:

```
no TODO → success
```

Но oracle всё ещё scenario-specific и не доказывает general semantic equivalence для произвольного real project.

SeededVariantGenerator тоже хорошая основа, но пока не determinism stress system.

Generator:

- принимает deterministic seed;
- строит pairwise rows;
- использует deterministic ordering functions;
- сохраняет content hashes;
- пишет generated scenario metadata.

`LabMetamorphicAnalyzer` сравнивает expected status, quality, oracle и diagnostic/outcome signatures.

Это направление очень близко к тому, что нужно. Современная работа именно по metamorphic testing transpilers также исходит из того, что обычный exact-output oracle для transpiler сложен и полезно формулировать отношения между исходным и transformed input/output; опубликованный в 2026 году preprint по metamorphic testing transpilers рассматривает mutation-consistency именно как средство поиска ошибок, которые простое fuzzing не обнаруживает. Это не следует копировать буквально, но подход хорошо совпадает с задачей Migrator. ⁵

Почему большое количество тестов не гарантирует production reliability.

Текущий test portfolio доказывает разные узкие свойства.

Тип	Что доказывает	Чего не доказывает
unit	локальная функция для известных shapes	interaction/state
recognizer test	recognizer понимает fixture syntax	project symbol correctness
config contract	schema/selected examples	ambiguity всей configuration
renderer snapshot	output стабилен относительно fixture	semantic correctness
golden master	output не изменился	output правильный
StandardMigrationMode contract	expected architectural strings существуют	runtime artifact identity
project fixture	выбранный small project проходит	real topology/helpers
Lab stable	curated scenarios сохраняют contract	unseen interactions
semantic oracle	scenario-specific behaviour	universal equivalence
metamorphic	invariants для generated variants	transformations, которых generator не создаёт
saved seed	конкретный regression не вернулся	соседние state spaces

Тип	Что доказывает	Чего не доказывает
release gate	declared corpus/evidence constraints	exact production artifact provenance в текущей модели

Snapshot problem.

Если compiler стабильно генерирует неправильное:

```
await Page.GetByText("Save").ClickAsync();
```

вместо semantically correct target, golden snapshot будет защищать ошибку от изменения.

Поэтому preferred oracle order:

```
semantic/runtime oracle
>
typed IR invariant
>
compiler verification
>
structural source assertion
>
snapshot
```

Snapshot полезен как change detector, не как correctness proof.

Synthetic simplification problem.

Fixture:

```
driver.FindElement(By.Id("save")).Click();
```

может отлично мигрировать.

Real project:

```
SaveButton.Click();
```

где:

```
SaveButton
→ inherited PageObject property
→ generic component
```



```
→ extension method
→ cached locator
```

имеет тот же business intent, но совершенно другой semantic graph.

Текущий single-file frontend теряет именно эту dimension.

Missing interaction problem.

Каждый feature отдельно может иметь test:

```
PageObject
async wait
generic helper
base fixture
```

Но production failure возникает в их composition:

```
inherited generic PageObject
→ helper returning collection
→ async assertion
→ override lifecycle boundary
```

Pairwise generation полезна, но для compiler-like systems нужны ещё guided multi-feature combinations.

State weakness.

Lab в основном создаёт clean workspaces.

Production сталкивается с:

```
failed previous run
partial output
stale report
old verify
continue
source modified
```

Поэтому clean Lab и dirty production проверяют разные systems.

Новый metamorphic catalogue.

Для semantic-preserving source transform **T** требуется:

$$Normalize(M(S)) = Normalize(M(T(S)))$$

не обязательно byte-equality generated code, но equality canonical migration semantics.

Добавить transformations:

Transformation	Expected invariant
shuffle source files	target semantic hash identical
shuffle unrelated members	identical semantic target
reorder <code>using</code>	identical
namespace block ↔ file-scoped	identical
whitespace	identical
LF ↔ CRLF	identical canonical result
variable extraction of locator	equivalent
helper extraction	equivalent intent
private method extraction	equivalent
harmless local rename	equivalent
alias Selenium namespace	equivalent
equivalent <code>By.Id</code> representation	equivalent
relocate base class	same symbols/effects
split partial class	equivalent
extension-method indirection	equivalent if symbol contract same
relocate source file	semantics same except explicitly path-scoped config
reorder config mappings	same result unless precedence explicitly declared

Для path-scoped config последний-but-one metamorphic relation должен учитывать, что relocation намеренно меняет semantic input `C(S)`.

Determinism stress harness.

Для каждого выбранного Lab scenario выполнять минимум `N=100` :

```
base source
base config
fixed migrator build
fixed package lock
fixed logical environment
```

На каждом run:

```
create clean workspace
randomize physical file creation order
randomize safe directory creation order
vary temp root pathname
optionally vary current working directory
run migration
canonicalize
hash
```

Canonical result:

```
CanonicalResult {
    generatedSemanticTreeHash,
    reportHash,
    sortedDiagnostics,
    sortedMappings,
    activeScopeSet,
    unresolvedSet,
    qualityClassification,
    verificationClassification
}
```

Не включать в semantic hash:

```
timestamps
absolute temp paths
GUID run id
elapsed time
```

Но отдельно тестировать raw artifact reproducibility, если это product requirement.

Для byte-reproducible target:

```
normalize newline policy = LF
UTF-8 no BOM policy explicit
stable source ordering
stable generated naming
stable formatting
```

При divergence:

```
save:
    seed
    source snapshot
    config snapshot
```

```
environment
all distinct canonical hashes
smallest diff
```

Официальный .NET contract делает file-order perturbation особенно важным: filesystem API не предоставляет тот стабильный порядок, на который production code сейчас частично рассчитывает. ²

Convergence harness.

Для initial failure:

```
X0 → X1 → ... → Xn
```

сохранять:

```
{
  "stateHash": "...",
  "targetHash": "...",
  "defectVector": {
    "internal": 0,
    "compile": 5,
    "semantic": 1,
    "unsupported": 4,
    "unmapped": 3,
    "todo": 2,
    "warnings": 7
  },
  "candidateFingerprint": "...",
  "patchHash": "..."
}
```

Assertions:

```
stateHash never repeats
accepted candidate strictly improves D
no higher-priority dimension regresses
candidate is applied transactionally
cycles <= configured bound
SUCCESS only through final gate
```

Если agent proposal не улучшает vector:

```
reject + rollback
```

Если нет candidate:

BLOCKED_UNKNOWN_SEMANTICS

Если state repeats:

REMEDIATION_CYCLE_DETECTED

Это превращает remediation из narrative process в testable algorithm.

Fault injection matrix.

Обязательные scenarios:

Fault	Oracle
process killed during write	next run uses no partial authoritative target
verification crash	no PASS evidence
partial generated dir	rejected or recreated
corrupt report	schema/hash failure
missing evidence	explicit EVIDENCE_MISSING
stale evidence	target hash mismatch
previous failed run	cannot contaminate new run
cancellation	manifest terminal CANCELLED ; no promotion
invalid config	fail before generation
duplicate rule	deterministic config error
ambiguous scope	deterministic config error
compiler missing	ENV_SDK_MISSING , not migration defect
browser missing	ENV_BROWSER_MISSING
source modified during run	snapshot/hash mismatch
target modified before verify	target hash mismatch
readonly path	explicit IO failure
spaces/Unicode path	same canonical output
Windows/Linux	documented E difference or canonical equality
huge project	bounded resources, same semantics
concurrent run same logical project	separate immutable run IDs, no state collision

Repro bundle.

Любой failure должен автоматически выдавать:

```
repro/  
  manifest.json  
  source/  
  config/  
  environment.json  
  tool.json  
  diagnostics.json  
  generated/  
  verification/  
  seed.json
```

Для большого source project source subset сначала может быть conservative dependency slice, затем автоматически минимизироваться при сохранении failure fingerprint.

Именно здесь стоит объединить нынешние:

```
evidence pack  
learn pack  
Lab triage  
saved seeds  
regression promotion
```

в один lifecycle:

```
production failure  
→ content-addressed repro  
→ reducer  
→ saved seed/project fixture  
→ regression promotion
```

Сейчас эти механизмы решают соседние задачи, но не образуют один строгий provenance chain.

Traceability matrix наиболее важных defects.

Problem → code → mechanism	Нарушенный invariant	Minimal repro	Oracle	Fix / disposition / test
unsorted files → <code>RoslynTestFileParser.ParseDirectory</code>	deterministic traversal	2+ files, permuted creation	equal canonical hash	sort normalized paths; FIX , E2E/ metamorphic

Problem → code → mechanism	Нарушенный invariant	Minimal repro	Oracle	Fix / disposition / test
duplicate class naming → <code>ResolveFileName</code>	source→target identity	two same class names	source/ output map equal	identity-based names; REPLACE , project fixture
old generated files → <code>RunMigrate</code> , <code>RunOrchestrate</code>	target=f(current input)	run with two outputs then one	tree contains only manifest files	staging+atomic snapshot; REPLACE , fault E2E
verify reruns pipeline → <code>RunOrchestrate</code>	verify exact T	inject permutation between passes	verified target hash == generated hash	generate once; DELETE repeated pass , E2E
verify-project regenerates	evidence bound to T	orchestration T1, verify T2	hash equality required	compile exact tree; REPLACE , E2E
final gate weak provenance → <code>check-final-gate.ps1</code>	no stale evidence	failed orchestration + old PASS verify	gate must fail	manifest/hash gate; REPLACE , contract+E2E
first prefix mapping → <code>DefaultProjectAdapter.ResolveTarget</code>	order independence	nested prefix mappings reordered	same resolution or ambiguity	longest-specific/typed; FIX , adapter property
multi-scope compatibility	unambiguous config	overlapping scopes reordered	always config error	remove first-match; DELETE , config
broken wildcard matcher	config semantics	<code>**/*.cs</code>	expected matches	canonical glob resolver; REPLACE , config
catch-and-skip parser	every source accounted	induced read/parser fault	typed file failure	no omission; DELETE fallback , frontend
first recognizer wins	unique semantics	syntactically overlapping receiver	ambiguity detected	candidate arbitration; REPLACE , recognizer property

Problem → code → mechanism	Нарушенный invariant	Minimal repro	Oracle	Fix / disposition / test
single-file SemanticModel	source semantics preserved	extension/helper/inheritance fixture	symbol-resolution oracle	real project compilation; REPLACE , integration
permissive quality defaults	PASS soundness	unsupported/TODO but syntax valid	migration cannot be Complete	tiered statuses + strict final policy; FIX , E2E
agent-defined PROGRESS	monotonic remediation	A↔B repairs	repeated state rejected	core metric/state hashes; MOVE OUT OF AGENT , convergence
legacy↔V2↔legacy	one canonical representation	round-trip property corpus	semantic field equality	direct V2 frontend/lowering; MERGE/DELETE , IR differential

WHAT SHOULD BE DELETED?

Ниже — не список «можно когда-нибудь почистить», а кандидаты, существование которых непосредственно увеличивает количество допустимых, но противоречивых states.

Удалить **multiple-scope compatibility mode** `first wins`.

Сейчас решает: старый config с overlapping scopes не падает.

Цена:

```
config array order
→ semantic migration result
```

Что ломается: intentionally ambiguous configs.

Чем заменить:

```
migration config error
+
automatic config diagnostic:
  File X matches scopes A and B
```


Если нужен inheritance — реализовать explicit hierarchical scopes.

Verdict: DELETE.

Удалить повторную миграцию из Verify stage.

Сейчас:

```
source → migrate → target A
source → migrate → verify target B
```

Она не добавляет информацию.

Что сломается: ничего conceptually; verifier нужно научить принимать persisted/in-memory immutable `TargetArtifact`.

Чем заменить:

```
TargetArtifact target = pipeline.Run(snapshot)
Write(target)
Verify(target)
```

Уменьшение state space очень большое:

```
A == B assumption
```

исчезает полностью.

Verdict: DELETE. P0.

Удалить повторную regeneration из `verify-project` как production evidence mechanism.

Сейчас она удобна как independent «can migrator regenerate something compilable?» test.

Это можно сохранить как **diagnostic/regression command**, но не как evidence того, что orchestration target скомпилировался.

Production verification:

```
verify-project --target-manifest <manifest>
```

и build exact files.

Verdict: REPLACE in production; optional diagnostic command KEEP.

Удалить latest-run resolution из final gate.

`latest` — UI convenience, не provenance.

Final gate должен требовать:

```
--manifest <exact-id>
```

Agent/user UI может определить latest для display, но deterministic gate — никогда.

Verdict: DELETE.

Удалить `AllowMissingVerification` из production-success semantics.

Можно оставить в local diagnostic mode:

```
status = NOT_VERIFIED
```

но не преобразовывать отсутствие proof в success.

Verdict: DELETE FROM PASS PATH.

Удалить broad catch-and-skip source parsing.

Каждый declared source file обязан дать:

```
Parsed  
или  
Failure
```

Нельзя «не заметить» файл.

Verdict: DELETE fallback; FIX result type.

Удалить duplicate scope matching implementation.

Adapter и VerifyRunner не должны иметь собственные реализации одной semantics.

```
ScopeResolver  
├ Adapter  
├ Validator  
└ Verify
```

Verdict: MERGE.

Удалить LegacyIrBridge **как steady-state architecture.**

Не немедленно `git rm`, а после direct canonical IR migration.

Особенно бессмыслен .NET path:

```
Legacy → V2 → Legacy
```

Его можно отключить раньше, чем завершится canonical IR migration.

Что сломается:

- experimental V2 parity tests;
- compatibility APIs;
- TypeScript paths, если они используют bridge input.

Переходный plan должен сначала сделать frontend V2-native и DotNet renderer V2-native.

Verdict: MERGE representations → DELETE bridge. P1.

Удалить caller/agent-owned PROGRESS **classification.**

Agent может сказать:

```
I expect this to fix X
```

но recorded progress должен вычисляться:

```
core.Compare(Xbefore, Xafter)
```

MetricSummary как свободный authoritative string не нужен.

Verdict: MOVE OUT OF AGENT / DELETE authority. P0.

Удалить non-canonical candidate fingerprints.

Fingerprint должен быть вычисляемым:

```
hash(  
  errorCode,  
  normalized symbol/location,  
  transformKind,  
  semantic parameters  
)
```

не придуманным формулировкой agent.

Verdict: REPLACE.

Удалить persistent state, которое является cache вычисляемой информации, из correctness path.

Например, если:

```
unmapped list  
quality summary  
candidate list
```

полностью вычисляются из source/config/target, они не должны быть independent source of truth.

Их можно сохранять как report/cache, но:

```
manifestHash mismatch → discard/recompute
```

Verdict: SIMPLIFY/DELETE authority.

Сильно сократить raw target-template configuration.

Не обязательно удалять весь escape hatch. Но обычная migration semantics не должна быть строковым mini-language.

Перевести:

```
Methods  
ParameterizedMethods  
TargetStatements  
TargetExpression
```

в typed mapping contracts там, где смысл известен.

Оставшийся raw mode:

```
unsafe  
explicit  
manual-review-required
```

и он не должен автоматически получать strongest PASS.

Verdict: REPLACE majority; KEEP narrow escape hatch.

Схлопнуть evidence pack / learn pack / triage / regression promotion вокруг единого ReproBundle schema.

Agent learning memory полезна как hints.

Она не должна существовать как параллельная truth model.

```
immutable evidence
↓
derived learning
```

а не наоборот.

Verdict: MERGE/SIMPLIFY.

Accidental complexity map.

Mechanism	Verdict	Причина
one CLI standard entry	KEEP	useful
<code>MigrationPipeline</code> basic parser→adapter→renderer abstraction	KEEP/FIX	хорошая локальная форма
repeated pipeline per orchestration stage	DELETE	duplicate state
source frontend registry	KEEP	legitimate source extensibility
syntax heuristics as semantic authority	REPLACE	insufficient
real Roslyn project semantics	ADD/REPLACE frontend	necessary complexity
legacy IR	DELETE after migration	duplicate representation
IR V2	KEEP/EVOLVE	лучший canonical candidate
<code>LegacyIrBridge</code>	DELETE	transitional only
config layers	KEEP/SIMPLIFY	useful explicit input
raw transformation DSL	REPLACE	weak typing
scopes	KEEP/FIX	valid concern
first-match scope compatibility	DELETE	ambiguity
duplicated scope matcher	MERGE	drift
mapping configuration	KEEP/SIMPLIFY	project-specific facts necessary
prefix first-match	FIX	order semantics

Mechanism	Verdict	Причина
TODO/unresolved diagnostics	KEEP	truthful partial migration
silent parser skip	DELETE	unsound
renderer	KEEP/REWRITE against canonical IR	necessary
VerifyRunner diagnostics	KEEP	useful V1/V3
VerifyRunner as success proof	REPLACE	insufficient
exact target compiler verification	KEEP/MAKE AUTHORITATIVE	necessary
independent re-generation in verify	DELETE	evidence mismatch
final gate	KEEP/REPLACE	concept correct, evidence weak
latest-run behaviour	DELETE	state sensitivity
autonomy bounded loop	SIMPLIFY	useful only after formal metric
agent progress judgment	MOVE OUT OF AGENT	core-decidable
agent semantic adaptation	KEEP IN AGENT	genuinely hard
Lab	KEEP/EXPAND	strong foundation
seeded/metamorphic generator	KEEP/EXPAND	directly useful
contract tests based on source text	SIMPLIFY	architecture lint, not correctness
regression promotion	KEEP/MERGE with repro bundle	valuable
multiple memory ledgers	SIMPLIFY/MERGE	too many truth-like states

Целевая архитектура, альтернативы, приоритеты и migration plan

Сравнение архитектур.

Критерий	A: Current enhanced	B: Deterministic compiler	C: Compiler + repair	D: Agent-heavy
correctness	3	5	5 при строгом acceptance	2-3

Критерий	A: Current enhanced	B: Deterministic compiler	C: Compiler + repair	D: Agent-heavy
determinism	3 после fixes	5	5 deterministic repairs / 4 with LLM proposals	1–2
implementation simplicity	2	4 после cutover	3	3 initially, 1 operationally
testability	3	5	5	2
handling real helpers	3	4	5	5
reproducibility	3	5	5	2
agent token usage	3	5 low	4	1
extensibility	4	4	5	4
support cost	2	5	4	2
ability to prove termination	2	5	4–5	1
ability to isolate failure	3	5	5	2

A — сохранить current enhanced pipeline.

Возможно стабилизировать, но придётся:

- сделать single immutable target;
- очистить state model;
- исправить ordering;
- заменить final evidence;
- формализовать remediation;
- унифицировать scope resolution;
- усилить frontend semantics.

После этого значительная часть нынешней architecture всё равно исчезнет.

То есть A постепенно превращается в B.

B — simplified deterministic compiler.

Наилучший base architecture:

```

snapshot
→ semantic frontend
→ typed IR
→ whole-project analysis
→ deterministic transforms

```

- lowering
- immutable target
- exact verification
- unresolved list

Главный недостаток — complex custom helpers иногда останутся unresolved.

Это приемлемо согласно главному критерию проекта.

C — compiler + repair loop.

Лучшее дополнение к B, если repair остаётся typed:

- compile errors
- typed diagnostics
- deterministic fix rules
- fixed point

Например:

- CS0246 known missing using
- deterministic add import
- known fixture signature mismatch
- deterministic lifecycle transformation

Unknown cases:

- agent proposal
- isolated apply
- deterministic acceptance

Это фактически рекомендуемая модель.

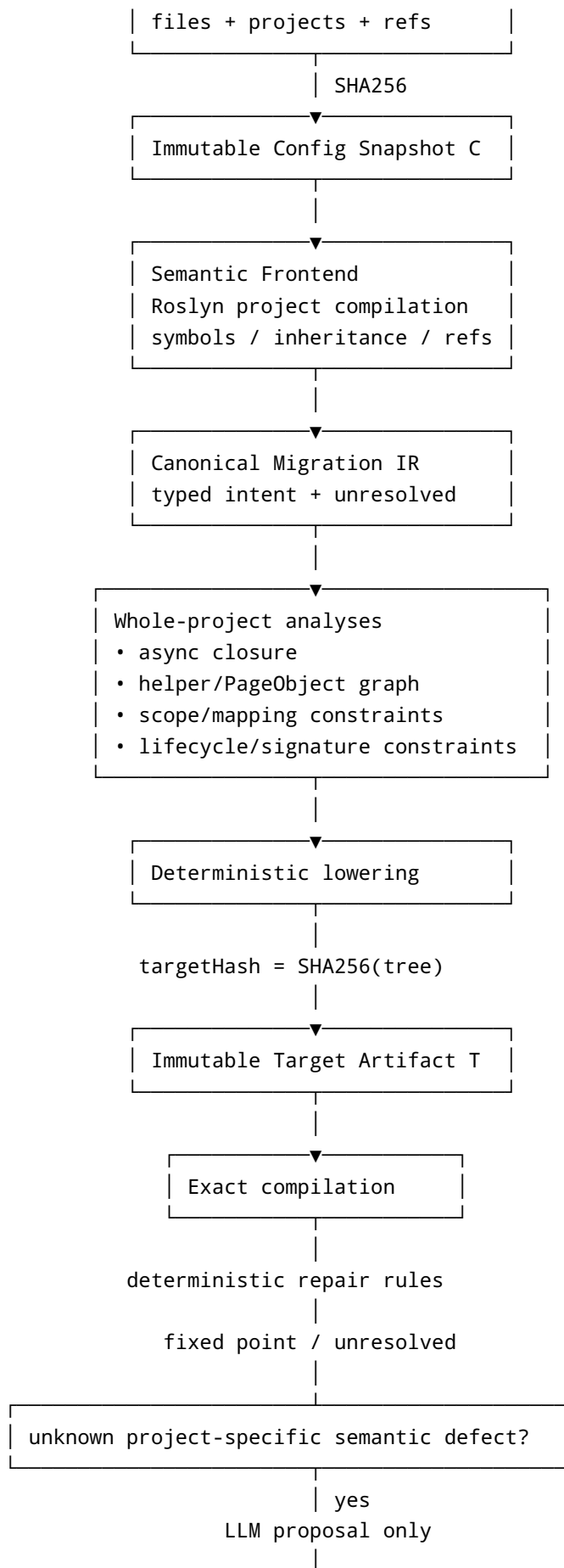
D — agent-heavy.

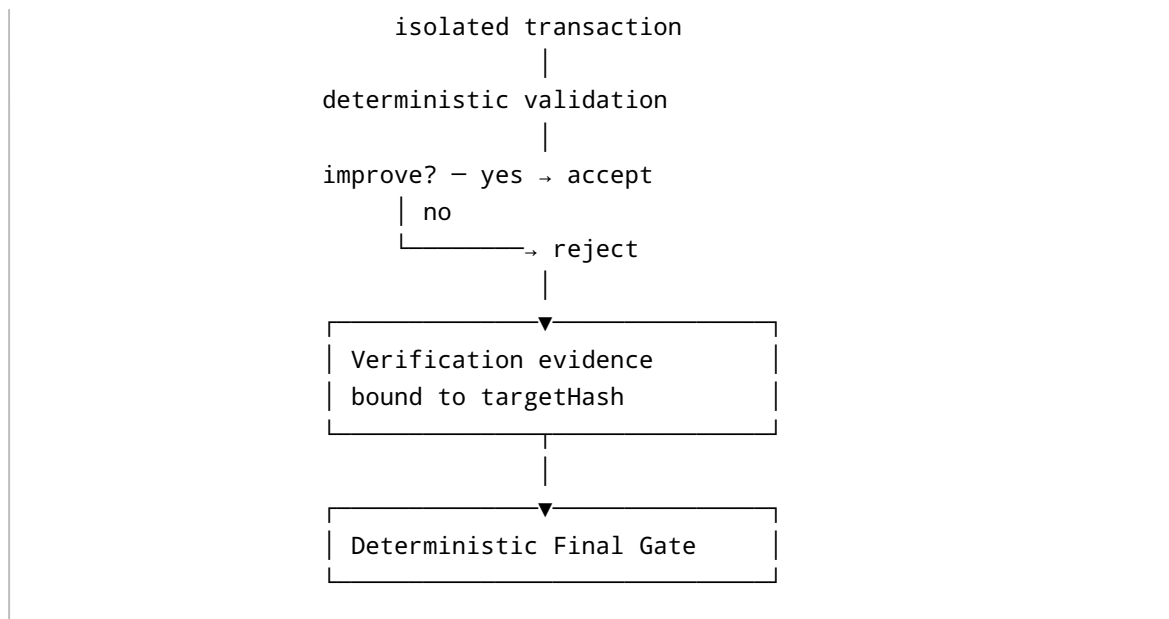
Лучше всего исследует неформализованный project-specific space, но хуже всего подходит как source of truth.

Её следует использовать не как architecture, а как **fallback search backend**.

Рекомендуемая architecture: B + ограниченный C.

| Immutable Source Snapshot S |





Это следует тому же принципу, который делает reproducible build systems надёжными: correctness зависит от полного набора inputs, execution graph и exact outputs, а не от интерпретации «примерно подходящих» artifacts прошлых запусков. ⁶

Prioritization.

Оценки: correctness/determinism/simplicity — benefit 1-5; cost/risk — выше означает дороже/рискованнее.

Change	Priority	Correctness	Determinism	Simplicity	Cost	Regression risk
single immutable target + verify exact tree	P0	5	5	5	3	3
content-hash run manifest/evidence	P0	5	5	4	3	2
clean/staging output, no stale files	P0	5	5	4	2	2
deterministic sort/naming/mapping resolution	P0	4	5	4	2	2
remove multi-scope first-match	P0	5	5	4	1	2
parser must account for every source file	P0	5	3	4	2	2

Change	Priority	Correctness	Determinism	Simplicity	Cost	Regression risk
core-owned progress/state hashes	P0	5	5	4	3	3
real project-level Roslyn semantic frontend	P1/P2	5	4	3 long-term	5	5
canonical IR; remove bridge/legacy	P1	4	4	5	5	5
typed config transformations	P1	5	4	5	4	4
async fixed-point graph	P2	5	4	3	5	4
expanded metamorphic/state Lab	P1	4	5	3	3	1
deterministic compile repair	P2	4	4	3	3	3
performance/parallelism	P3	1	1 unless careless	1	2	4

Топ-10 в порядке реализации.

Первое. Сделать `MigrationRun` immutable: source snapshot, config snapshot, generated tree, hashes.

Второе. Удалить повторный migration из orchestration Verify и проверять exact Migrate target.

Третье. Переделать `verify-project` на exact target tree + hash-bound evidence.

Четвёртое. Переделать final gate на manifest-based verification; удалить latest/missing-verification success semantics.

Пятое. Устранить все semantic order dependencies: sort files, deterministic names, unique/longest mappings, scope ambiguity errors, fully sorted reports.

Шестое. Удалить catch-and-skip source semantics: каждый source file должен присутствовать в result.

Седьмое. Перенести progress/termination authority из agent в deterministic core; добавить state hashes/cycle detection.

Восьмое. Начать project-level semantic frontend на реальном Roslyn compilation, начиная с symbol index/helper graph без немедленной замены всего renderer.

Девятое. Ввести canonical typed IR и direct .NET lowering; затем удалить legacy bridge.

Десятое. Перенести standard semantic transformations из string config в typed lowering и усилить metamorphic/property corpus.

Plan A — Minimal stabilization.

Цель — максимально сохранить текущий код.

Последовательность:

1. Canonical sort all production filesystem traversal.
2. Deterministic ResolveFileName.
3. Make generated directory recreated/staged per run.
4. Pipeline executes once per orchestration invocation.
5. VerifyRunner receives exact MigrationResult.
6. verify-project builds exact generated files.
7. Add source/config/target hashes.
8. Make final gate require matching hashes.
9. Remove multiple-scope compatibility.
10. Longest-prefix or ambiguity for mappings.
11. Replace parser skip with explicit failure.
12. Make default completion policy strict.
13. Core computes remediation metrics.
14. Detect repeated state.
15. Add determinism/fault-injection Lab suite.

После этого текущая architecture станет существенно надёжнее даже без полной IR rewrite.

Plan B — Simplification — рекомендуемый.

Главная цель именно удалить state и duplicated mechanisms.

Step A

Introduce RunManifest + TargetArtifact while old code still works.

Step B

Change RunOrchestrate:

- one pipeline execution
- immutable result
- stages become views over same result.

Step C

Move all verification to TargetArtifact.
Delete re-migration in Verify.

Step D

Change verify-project to consume TargetArtifact.
Delete independent generation from evidence path.

Step E

Replace final-gate script semantics with manifest verifier.
Keep old script wrapper temporarily.

Step F

Create ScopeResolver + MappingResolver.
Route adapter/verifier through them.
Delete duplicated matchers and compatibility first-match.

Step G

Create core RemediationState:
 defect vector
 state hash
 candidate hash.
Agent becomes proposer.
Delete agent-owned PROGRESS authority.

Step H

Introduce Canonical IR alongside legacy,
but frontend populates canonical nodes directly for migrated feature slices.

Step I

Create direct DotNet canonical renderer.
Differentially compare with legacy in Lab.

Step J

Move feature families one at a time.
When canonical coverage reaches production corpus,
freeze legacy feature development.

Step K

Delete LegacyIrBridge + legacy production model.

Step L

Deprecate string statement mappings;
replace standard cases with typed transforms.

Step M

Collapse evidence/learn/triage state around ReproBundle + RunManifest.

Step N

Delete compatibility contracts whose only purpose was preserving removed
runtime architecture.

Каждый step оставляет repository working; важный принцип — **сначала invariant/test, затем reroute, только затем delete.**

Plan C — Clean-sheet Migrator 2.0.

Если backward compatibility не требуется, я бы вообще не строил нынешнюю adapter/action architecture.

Основные interfaces:

```
ProjectSnapshot Capture(ProjectInput input);

SemanticProject Analyze(
    ProjectSnapshot source,
    ConfigurationSnapshot config);

MigrationPlan Plan(
    SemanticProject project,
    ConfigurationSnapshot config);

TargetArtifact Lower(MigrationPlan plan);

VerificationEvidence Verify(
    TargetArtifact target,
    VerificationPolicy policy);

RemediationProposal[] ProposeUnknowns(
    MigrationPlan unresolved);

RunDecision Decide(
    RunManifest run);
```

`SemanticProject` содержал бы:

```
symbols
types
methods
calls
inheritance
framework lifecycle
locator facts
PageObject graph
source effects
tests
```

`MigrationPlan`:

```
resolved intents
unresolved intents
constraints
```

```
async closure
target structure
```

Ни одного `RawExpression` в verified core path.

Unknown:

```
UnresolvedSemantic {
  stableCode
  sourceSpan
  symbol
  requiredKnowledge
}
```

LLM adapter:

```
UnresolvedSemantic
+ bounded project slice
→ proposed SemanticRule / patch
```

Но proposal никогда не меняет run status непосредственно.

Минимальный regression suite после redesign.

Обязательные permanent tests:

```
unit:
  glob resolution
  mapping specificity
  canonical path identity

recognizer:
  ambiguity detection
  false-positive Click/custom API cases

semantic frontend:
  aliases
  extension methods
  inheritance
  generics
  partial classes
  multi-class files

IR:
  no Raw in verified nodes
  serialization determinism
  stable symbol identity
```

```

whole project:
  async transitive closure
  lifecycle boundaries

renderer:
  stable ordering
  naming collision independence

project fixture:
  PageObject + helpers + inheritance + waits

state:
  dirty prior run
  cancelled run
  source mutation
  stale evidence

metamorphic:
  file/member/using ordering
  formatting
  namespace shape
  helper extraction

determinism:
  100-run stress

convergence:
  no repeated state
  lexicographic monotonicity

end-to-end:
  exact-target compile
  exact-target evidence hash
  final gate provenance

```

Что существующие tests принципиально не доказывают.

Они не доказывают утверждение:

$$\forall S, C, E : M(S, C, E) \text{ sound and deterministic}$$

И не должны.

Но пока отсутствуют tests именно для следующих production-risk classes:

```

dirty workspace
permuted filesystem
artifact identity
source/config mutation between stages

```



```
ambiguous recognizer arbitration  
project-wide helper semantics  
async call graph  
agent state oscillation  
evidence mismatch
```

система имеет огромную область поведения за пределами доказанного corpus.

Минимальный набор runtime artifacts для подтверждения оставшихся гипотез.

Для следующего реального problematic run достаточно сохранить:

```
source snapshot hash + exact source archive  
merged normalized config  
tool commit/build identity  
OS + dotnet --info  
package lock/project files  
orchestration report  
generated tree  
verify report  
verify-project generated tree  
project-verify report  
final-gate evidence  
autonomy-state before/after each cycle  
agent candidate fingerprint/result  
compiler stdout/stderr
```

Первый эксперимент после этого должен быть не «ещё раз запустить agent», а:

```
hash orchestration generated tree  
hash verify-project generated tree  
compare
```

Если они различны — это прямое runtime подтверждение уже найденного static evidence-integrity defect.

Второй:

```
take exact same source/config  
100 clean migrations  
compare canonical hashes
```

Третий:

```
repeat with randomized file creation order
```

Четвёртый:

```
run → intentionally leave stale generated file → rerun
```

Пятый:

```
run remediation with state hashes  
look for X_i == X_j
```

Итоговая оценка доверия.

Сегодня Migrator можно рассматривать как:

полезный best-effort migration engine с развитой диагностической/Lab инфраструктурой, но не как доказуемо воспроизводимый migration compiler.

Самые сложные проблемы Selenium→Playwright действительно являются необходимой сложностью:

- project semantics;
- PageObjects/helpers;
- asynchronous propagation;
- custom DSLs;
- runtime equivalence.

Но значительная часть текущей нестабильности к этой сложности отношения не имеет. Она создана архитектурой:

```
multiple passes  
+ multiple truths  
+ multiple persisted states  
+ order-sensitive fallbacks  
+ weak evidence provenance  
+ heuristic agent progress  
+ dual representations
```

Поэтому стратегически неправильным было бы сейчас добавлять новые recognizers, новые recovery rules или ещё один orchestration layer.

Правильный следующий этап:

сначала сократить количество возможных состояний системы и создать единственную цепочку identity от source snapshot до verified target artifact.

После этого migration coverage можно снова наращивать.

Главная архитектурная цель должна быть сформулирована жёстко:

Тогда:

- filesystem permutation перестанет менять semantics;
- stale output не сможет стать current output;
- verifier не сможет проверить другое дерево;
- final gate не сможет принять старое evidence;
- scope ambiguity не сможет превратиться в first-match;
- recognizer uncertainty станет явным unresolved fact;
- agent не сможет объявить субъективный progress;
- remediation cycle станет обнаружимым;
- runtime failure станет воспроизводимым bundle;
- успешная миграция станет не интерпретацией нескольких reports, а проверяемым состоянием.

Именно такая архитектура реализует сформулированный в запросе приоритет:

**детерминированные 80% + честные, структурированные 20% unresolved
лучше, чем недоказуемые 95%.**

Причём переход к project-level semantic frontend, canonical typed IR и transactional agent proposals даёт возможность впоследствии увеличивать эти 80% без возвращения к недетерминированной orchestration complexity.

1 3 Hermeticity

https://bazel.build/basics/hermeticity?utm_source=chatgpt.com

2 Directory.GetFiles Method (System.IO)

https://learn.microsoft.com/en-us/dotnet/api/system.io.directory.getfiles?view=net-10.0&utm_source=chatgpt.com

4 Generate JavaScript with Roslyn and T4 Templates

https://learn.microsoft.com/en-us/archive/msdn-magazine/2016/december/roslyn-generate-javascript-with-roslyn-and-t4-templates?utm_source=chatgpt.com

5 Metamorphic Testing of Transpilers via Mutation Consistency of Programs

https://arxiv.org/abs/2607.29247?utm_source=chatgpt.com

6 Bazel Glossary

https://bazel.build/reference/glossary?utm_source=chatgpt.com